INDEX

120 PAGES

Name : ADITYA S Std. : Sec. :

Roll No. : Sub :

S.No.	Date	Title	Page No.	Teacher's Sign / Remarks
1.	27.7.23	Study of Linux and Windows Commands.	1-6	28/7/23
2.	04/08/23	Study of Different Types of Network Cables	7-9	04/8/23
3.	04/08/23	Experiment on Cisco Packet Tracer	10-12	11/8/23
4.	08/08/23	Setting up a Local Area Network [LAN]	13-15	15/8
5.	15/08/23	Packet Capture Tool: Wireshark	16-18	18/8
6.	01/09/23	Error Correction at Data Link Layer	19-21	7/9
7.	01/09/23	Flow control at Data Link Layer using Sliding Window Protocol.	22-24	8/9
8.	08/09/23	NMAP to Discover the Live Host using Nmap Scans on the TryHackme Platform	25-27	8/9

9. 22/09/26 Implementation of Subnetting in CISCO PACKET TRACER Simulator (9) 22/9

12. 29/09/26 End-To-End Communication at Data Link Layer. (9) 29/9

13. 30/09/26 Implementing own PING Program (9) 30/9

14. 01/10/26 Packet Sniffing using Raw Source. (9) 01/10

10. 08/10/26 Internetworking with Router using CISCO Packet Tracer (9) 8/10

11. 08/10/26 Static Routing and RIP Dynamic Routing. (10) 8/10

completed

8/10/26

EX NO: 1

27/07/2026

Study of Network Commands

Windows Command

1. arp -a : ARP is a short form of address Resolution Protocol

OUTPUT:

Interface : 10.16.86.50 ... 0x9

Internet Address	Physical Address	Type
10.16.56.155	ec-12-17-17-12-5c	Dynamic
10.16.56.255	ff-ff-ff-ff-ff-ff	Static.

2. hostname : This is the simplest of all TCP/IP command. It will display the name of your computer

OUTPUT:

DESKTOP-DA9QFME

3. ipconfig /all : This command displays the configuration information about your TCP/IP connection

OUTPUT:

Windows IP configuration

Hostname DESKTOP-DA9QFME

Node Type .... Mixed.

4. nbtstat -a: This command helps solve the problem with NETBIOS name resolution.

OUTPUT:

Ethernet 2:

Node IP address : [192.168.66.1] Scope Id: []

Host not found.

5. netstat : netstat displays a variety of statistics about computer Active TCP/IP connection.

OUTPUT:

Proto	Local Address	Foreign Address	State
TCP	10.16.56.50:49444	4.213.25.242:https	Established

6. nslookup : It is used to display DNS details such as IP address of Particular computer.

OUTPUT :

Server : Unknown

Address : 10.16.56.155

7. Pathping : It is a unique window cmd and its basically a combination of the ping and tracer cmd.

OUTPUT: cmd : pathping -g google.com

Tracing route to google.com [142.250.66.14] over a max of 30 hops:

8. ping : It is used to check whether one device reach another

OUTPUT: Ping google.com

Pinging

9. Route :

It is used to show/manipulate the IP routing Table . It is primarily used to setup static routes to specific host on network via host.

Manipulate network Routing Table

Route : [-f] [-p] [-4] [-6] command

13... 00 00 27 00 00 00 ... virtual box only

=> Linux Commands

1. IP commands

i) cmd: ip address show - display all IP address of network Interface.

OUTPUT:

eth0: inet 192.168.1.10/24

ii) ip address add 192.168.1.254/14 - Assign an ip address to an interface

OUTPUT: IP address added successfully

iii) ip link set eth0 up - Enables the network Interface

OUTPUT: eth0 state up

iv) ip link set eth0 down - Disables the network interface

OUTPUT: eth0 state down

v) ip link set eth0 promise on: - Enables Promiscuous mode

O/P: Promiscuous mode enabled

vi) IP route add default via 192.168.1.254 dev eth0 - Adds the default gateway

vii) ip route get 10.10.1.4 - shows the route to a destination IP.

OUTPUT: 10.10.1.4 via 192.168.1.254 dev eth0

2. ifconfig:

Displays network interface configuration

OUTPUT:

eth0 link encap: Ethernet inet 192.168.1.10

3. mtr:

i) mtr google.com. Shows route statistics to get a host

OUTPUT: HOST: localhost loss% avg Best Wrst

ii) mtr -g google.com - Shows numeric IP address only

OUTPUT: 142.250.183.206

iii) mtr -b google.com - Shows both hostname and IP address.

OUTPUT: google.com [142.250.183.206]

iv) mtr -c 10 google.com : Sends only 10 Packet.

4. tcpdump

i) dnf install -y tcpdump - install tcpdump.

ii) tcpdump -D - List available capture interface

OUTPUT:

1. eth0.2.IO

iii) tcpdump -i eth0 -c 10 - capture packets to/ from a host.

OUTPUT: 10 Packets captured

iv) tcpdump -i eth0 src host 8.8.8.8 - Capture Packets from a host.

OUTPUT:

8.8.8.8 > 192.168.1.10

v) tcpdump -i eth0 dst host 8.8.8.8 - Captures Packets to a host.

OUTPUT: 192.168.1.10 > 8.8.8.8

vi) tcpdump -i eth0 port 53 - Captures DNS Traffic

OUTPUT: ~~TCP~~ DNS captured.

vii) tcpdump -i eth0 -c 10 host www.google.com/ and port 443 - capture HTTPS Products

OUTPUT: TCP 443 captured

viii) tcpdump -i eth0 port not 53 and not 25 - capture all traffic except DNS and SMTP

OUTPUT: TCP/IP Packets Displayed.

5. ping:

i) ping google.com - checks connectivity to a host

OUTPUT: 64 bytes from google.com time = 12ms

ii) ping -c 10 google.com - sends only 10 ping request.

OUTPUT: 10 Packets Transmitted, 10 received

6. nmcli

i) nmcli connection show - Lists all network connections

O/P: wired connection 1

ii) nmcli connection modify "wired connection 1" modifies an existing connection.

iii) nmcli connection show - Display connection details

O/P: IPU. method auto

iv) nmcli connection modify "wired connection 1" IPU4 method auto - configure DHCP

O/P: IPU4 method set to auto

v) nmcli connection modify "wired connection 1" IPU4. method manual IPU4. addresses 192.0.2.1/24 IPU4. gateway 192.0.2.254 IPU4. dns 192.0.2.200 - Configures a static IPU4 address

EX NO: 2
02/08/2026

Study of different types of Network cables

AIM:

Understand different types of network cables

Different Types of Cables used in networking are:

1. Unshielded Twisted Pair [UTP] Cable
2. Shielded Twisted Pair [STP] Cable
3. Coaxial Cable
4. Fibre Optic Cable

Cable Type	Category	Maximum Data Transmission	Advantages & Disadvantages	Applications
UTP	Category 3	10bps	Advantages • cheaper in cost • easy to install as they have a smaller overall diameter Disadvantage • More prone to electromagnetic interference and noise	10Base-T Ethernet
	Category 5	Upto 100 mps		Fast Ethernet Gigabit Ethernet
	Category 5e	1Gbps		Fast Ethernet, Gigabit Ethernet
STP	Category 6	10Gbps	Advantages • Shielded • Faster than UTP • Less acceptible to noise and interference Disadvantages • Expensive • Greater Installation effort	
	Category 7	10Gbps		

Cable Type	Category	Maximum Data Transmission	Advantages & Disadvantages	Application / Use
Coaxial	RG-6 RG-59 RG-11	10-100 Mbps	Advantages • High bandwidth • Immune to interference • Low Loss, Bandwidth • Versatile Disadvantage • Limited Distance • Cost • Size Bulky	Speed of Signal is low Television Network, High Speed Internet connections
Fibre optics cable	Single mode, Multi-mode	100 Gbps	Advantages • High Speed • High Bandwidth • High Security • Long Distance Disadvantages • Expensive • Require Skilled Installers	Maximum Distance of fibre optics cable is around 100m

* RESULT:

Thus, the different Types of network cables were studied and understood successfully

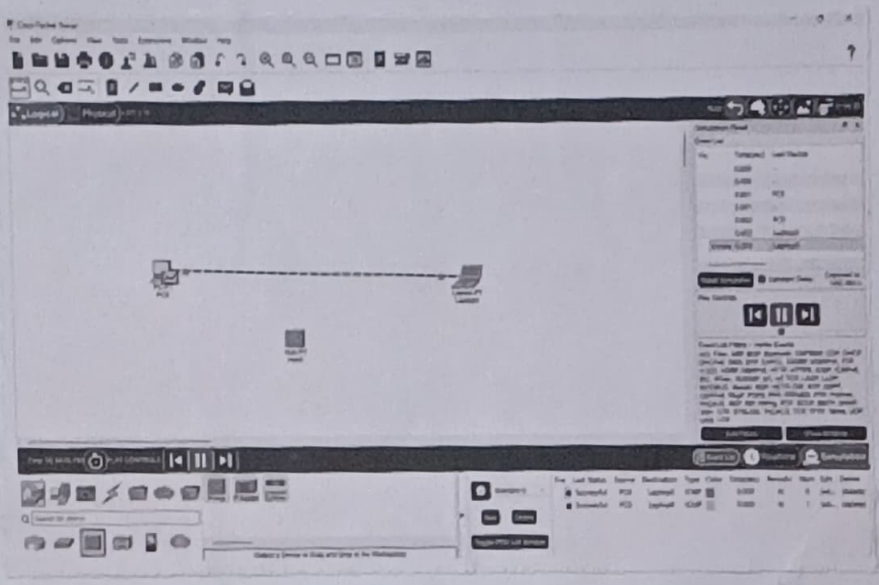
10/8/24

EXPT NO: 3 Experiment on Cisco Packet Tracer
01/08/2024

- AIM:

To understand its environment and behaviour of network devices of CISCO Packets Tracer Simulator.

1) Peer-to-Peer Connection (P2P):

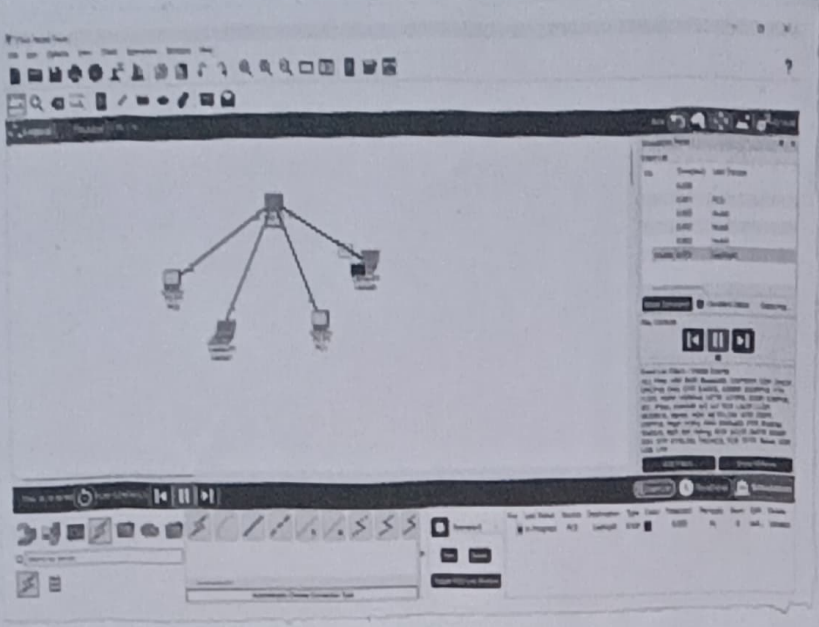


Cross-over cables were used to connect the PC and the Laptop

Only a cross-over cable can enables the communication directly between the same type and of end devices.

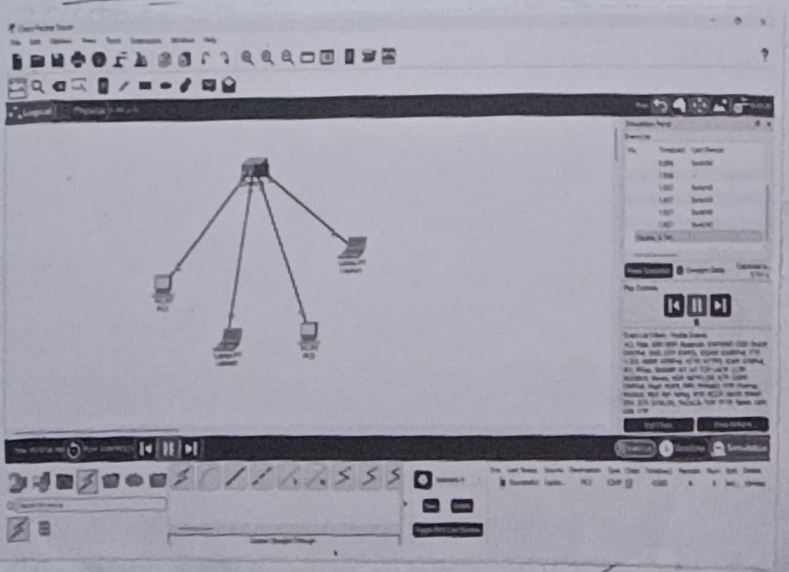
Usage of such cables have been replaced by switches, hubs and Routers.

2) Hub Connection



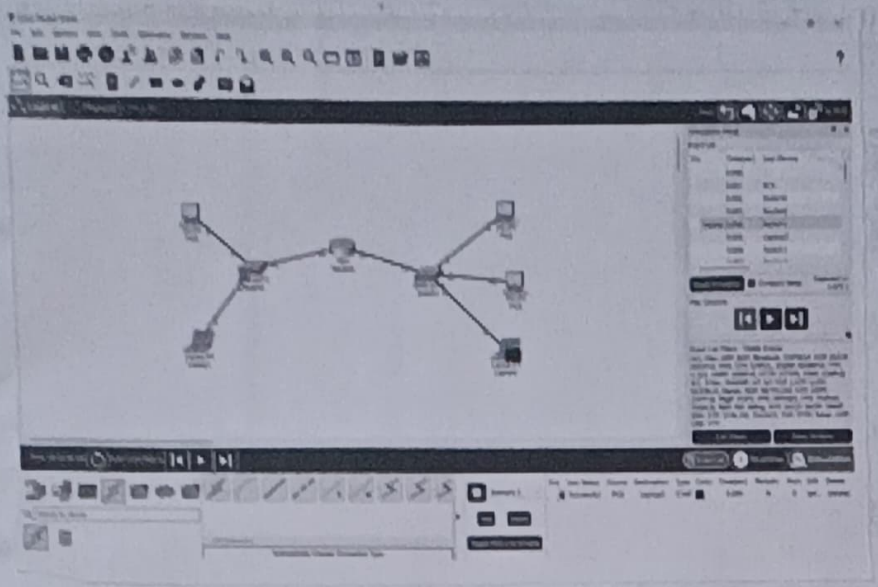
A hub connects multiple devices in a network by blindly broadcasting all incoming data signals to every connected devices.

3) Switch Connection.



Switch connects multiple devices within single LAN and intelligently towards data packets directly to specific destination using MAC addresses

4) Router connection :



A Router connects different subnets distinct networks together, directly data traffic between them using IP addresses and default gateways

* RESULT :

Hence the environment of CISCO PACKET TRACER and behaviour of the network devices in the software were studied, simulated and verified successfully.

10/8/26

EXPT NO: 4
03/05/2026

SPITTING UP A LOCAL AREA NETWORK [LAN]

- **AIM:**

To design, set up and configure a local area Network [LAN] connecting four computers through a network switch and demonstrate resource / data sharing across the network.

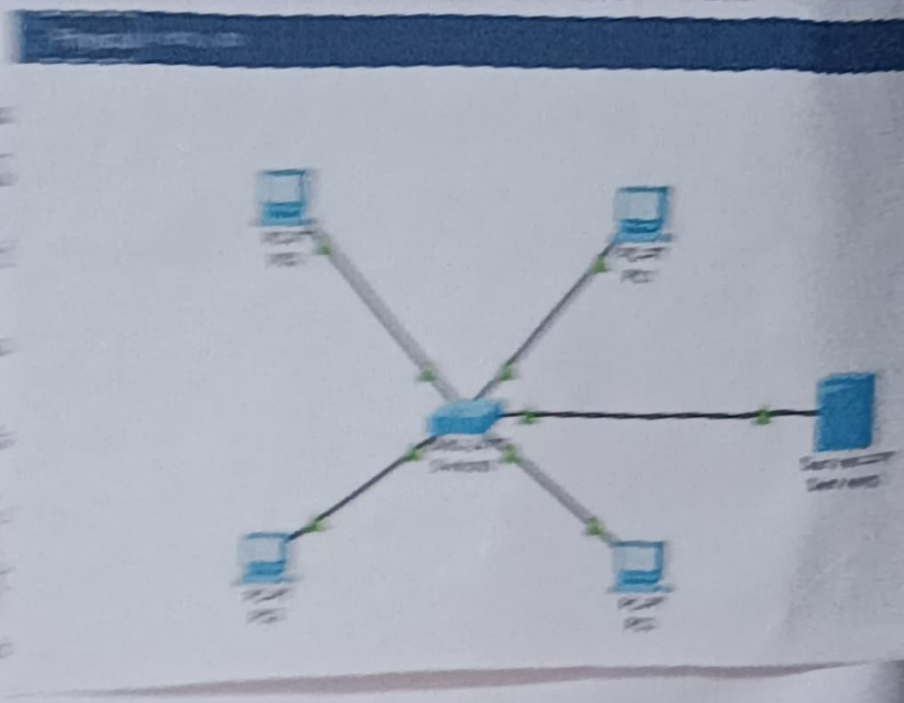
- **Equipment Required:**

Equipment	Quantity	Specification
Personal Computer	4	Ethernet [NIC] enabled
Network Switch	1	8/16/24 Port
Ethernet Cables [cat 5e / cat 6]	4	Straight through RJ-45 connectors

- **IP address configuration Table**

Device	IP Address	Subnet mask	Connected to
PC0	10.1.1.1	255.0.0.0	Switch Port Fa 0/1
PC1	10.1.1.2	255.0.0.0	Switch Port Fa 0/2
PC2	10.1.1.3	255.0.0.0	Switch Port Fa 0/3
PC3	10.1.1.4	255.0.0.0	Switch Port Fa 0/4
Switch	10.1.1.5	255.0.0.0	VLAN 1 Interface

Implementation : CISCO Packet Tracer



1) Place the Devices

→ Open CISCO Packet Tracer

→ From the device Category Panel. Select End Devices and drag 4PC onto the workspace

→ Select Switches and drag 1 Switch

2) Cable the Switch/Device:

→ Click Connections → Select either straight-through cables

→ Click each PC, then click the switch and select an available Fast Ethernet Port to complete each connection

→ Repeat for all four PC's Ports [Fa0/1 to Fa0/4]

Green Link Lights confirm a successful Physical connection.

3) Assign IP Addresses to PC's :

→ Click a PC - Desktop Tab - IP Configuration

→ Enter the Static IP Address and Subnet mask as per the IP Table.

4) Configure the Switch Management IP:

→ Click the Switch → CLI Tab

- enable
configure Terminal
Interface Vlan
IP Address 10.1.1.5 255.0.0.0
No Shutdown
exit.

1) Place the Devices

→ Open CISCO Packet Tracer

→ From the device Category Panel. Select End Devices and drag 4PC onto the workspace

→ Select Switches and drag 1 Switch

2) Cable the Switch/Device:

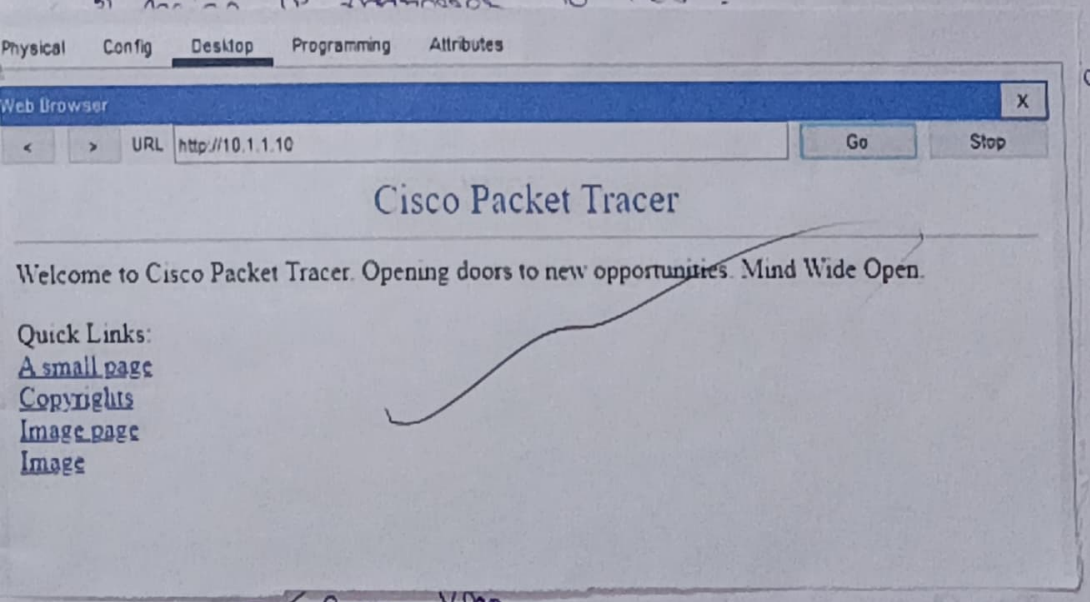
→ Click Connections → Select option straight-through cables

→ Click each PC, then click the switch and select an available Fast Ethernet Port to complete each connection

→ Repeat for all four PC's Ports [Fa0/1 to Fa0/4]

Green link Lights confirm a successful Physical connection.

3) Assign IP Addresses to PC's :



on

Interface Vlan

IP Address 10.1.1.5 255.0.0.0

No Shutdown

exit

5) Verify Connectivity using Ping

-> Click a PC -> Desktop Tab - and Prompt

-> Use Ping to Test connectivity

-> Successful Message

6) Demonstrate Data Sharing [Server Simulation]

-> Add a Server device [End Device -> Server -> PT]
and connect it to Switch

-> Assign it to an IP Address

-> Open the Server's service Tab & enables the HTTP

-> From another PC, open Web Browser and enter the server's IP address to load the Page confirmation. Successful data Transfer across LAN

. Result

This LAN is settled up and connection verified Successfully

EXPT NO: 5
19/8/26

PACKET CAPTURE TOOL : WIRESHARK

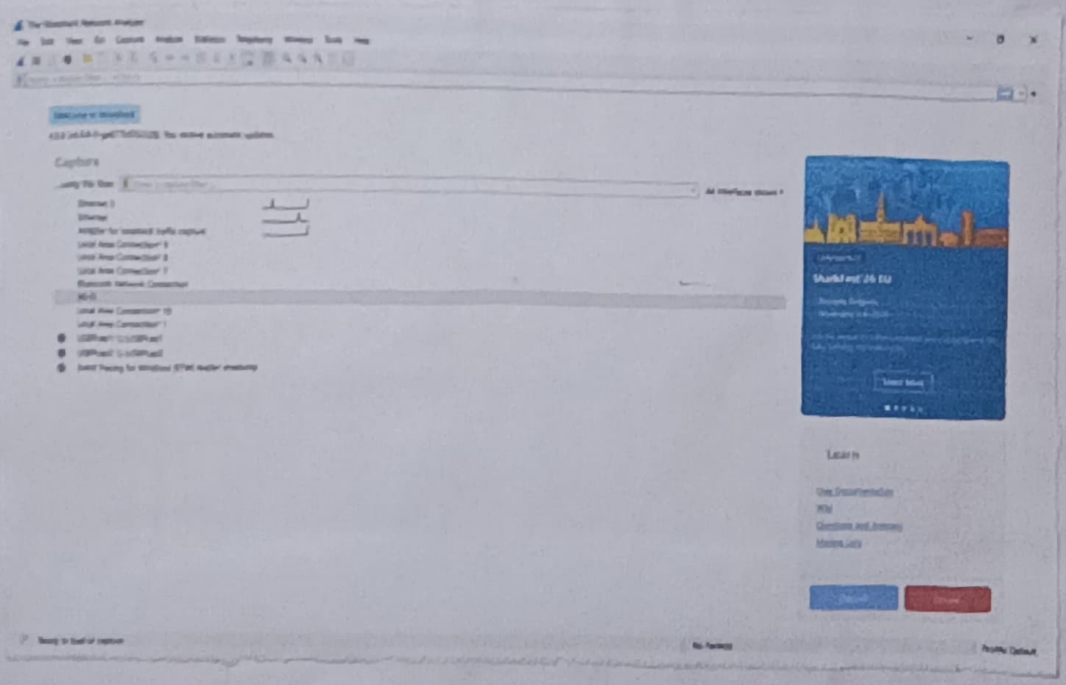
- **AIM:**

To understand the features of a wireshark as a packet capture tool and understand encapsulation of information at various layers of a Protocol Stack.

- **PROCEDURE**

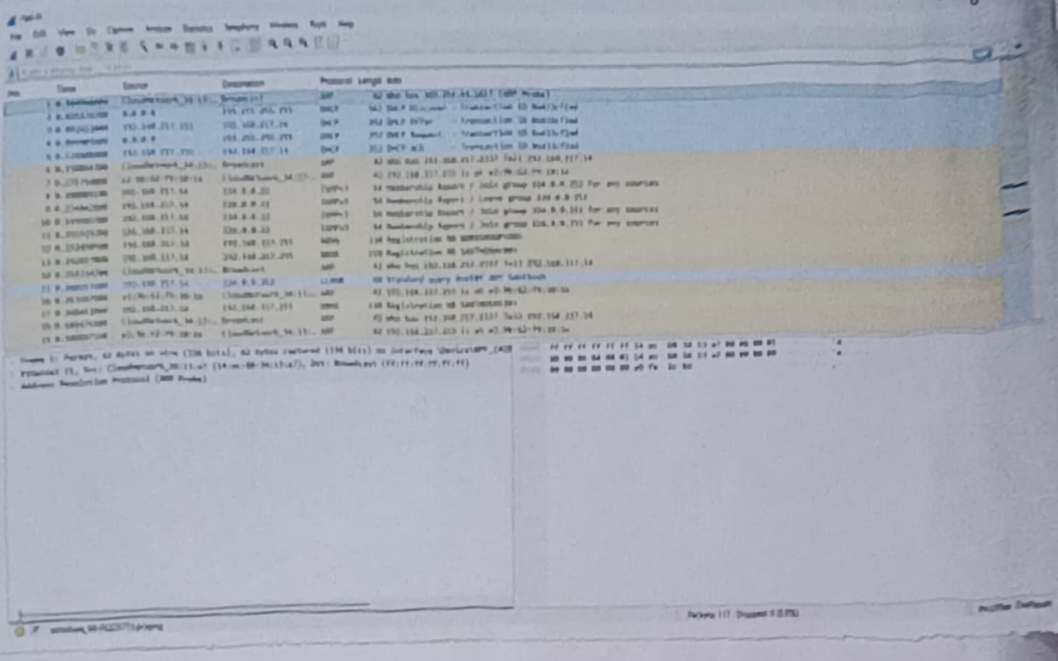
i) Open wireshark and select the wifi network Interface.

ii) Start the packet capture by double clicking the wifi - interface.



iii) Observe the Packet being captured in the wireshark window

iv) Different Packets such as ARP, DHCP, NDNS and LLMNR can be observed in the Packet List.

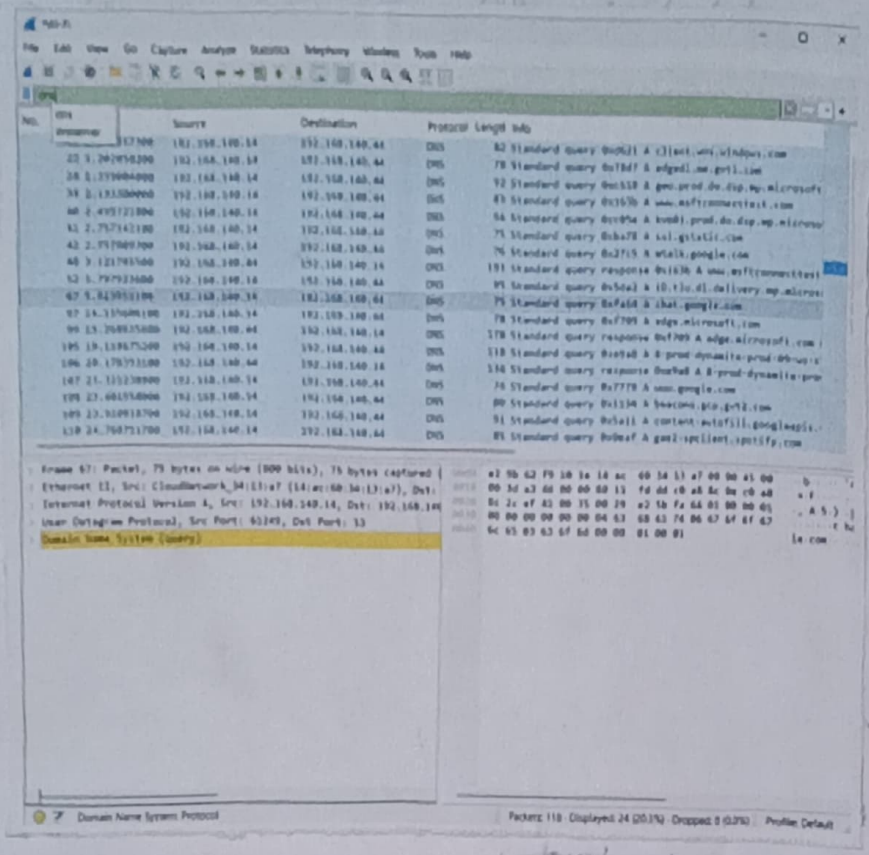


v) Select any Packet to view its detailed information such as frame, ethernet, UDP/TCP and Protocol Details.

vi) Apply the display filter in the filter bar to display only DNS Packets.

vii) Observe the DNS query and beyond Packets. The source and destination IP Address, Protocol, Packet Length, and DNS Information can be analysed.

viii) Stop the Packet capture other collecting the required Packets



* RESULT:

Thus, Network Packets were successfully captured and analyzed using wireshark and DNS Packets were identified using the display filter.

15/8/26

EXP.NO: 6
7/9/26

ERROR CORRECTION AT DATA LINK LAYER

- **AIM:**

To write a program to implement error detection and correction using Hamming code. Make a test run to input data stream and verify error correction feature.

- Sender.py:

```
def find_r(k):
    r = 0
    while (2**r < k+r+1):
        r+=1
    return r
text = input("enter text: ")
binary = ""
for ch in text:
    binary += format(ord(ch), "08b")
print("Binary:", binary)
k = len(binary)
r = find_r(k)
print("Data bits:", k)
print("Redundancy bits:", r)
n = k+r
code = [0]*(n+1)
s = 0
for i in range(1, n+1):
    if i & (i-1) != 0:
        code[i] = int(binary[j])
        j+=1
```

```
p = 1
while p ≤ n:
    parity = 0
    for i in range (p, n+1):
        if i≠p:
            parity = code[i]
    code [p] = parity
    p = p * 2
hamming_code = ""
for i in range (1, n+1):
    hamming_code += str (code[i])
print ("Hamming Code :", hamming_code)
with open ("Channel.txt", "w") as f:
    f. write (str[k] + "\n")
    f. write (hamming_code)
print ("Data saved in Channel.txt")
```

* Receiver.py :

```
def binary_to_text (binary):
    text = ""
    for i in range (0, len (binary), 8):
        byte = binary [i : i+8]
        if len (byte) == 8:
            text += chr (int (byte, 2))
    return text
with open ("Channel.txt", "r") as f:
    k = int (f. readline ())
    data = f. readline(). strip()
print ("Received Hamming Code", data)
```

```
n = len(data)
code = [0] + [int(x) for x in data]
corrupted_binary = ""
for i in range(1, n+1):
    if i*(i-1)!=0:
        corrupted_binary += str(code[i])
corrupted_binary = corrupted_binary[k]
corrupted_text = binary_to_text(corrupted_binary)
print("corrupted_text:", corrupted_text)
error = 0
p = 1
while p<=n:
    parity = 0
    for i in range(p, n+1):
        if i&p:
            parity ^= code[i]
        if parity:
            error += p
        p = p*2
if error==0:
    print("No error detected")
else:
    print("error detected at position", error)
    code[error] = 1 - code[error]
    print("error detected")
corrected_binary = ""
for i in range(1, n+1):
    if i&(i-1)!=0:
        corrected_binary += str(code[i])
```

corrected_text = binary_to_text(corrected_binary)

print("corrected_text", corrected_text)

* RESULT:

Thus, error detection and correction done using Hamming code.

EXPT NO: 7

8/9/26

FLOW CONTROL AT DATA LINK LAYER USING SLIDING WINDOW PROTOCOL

- AIM:

To write a program to implement flow control at data link Layer using sliding window Protocol, Simulate the flow of frames from one node to another.

* Create a Sender Program with following features:
  i) Input window size from the user
  ii) Input a text message from the user.
  iii) Consider one character per frame.
  iv) Create a frame with following fields
  v) Send the frames [print output on screen & save it in a file called sender-buffer]
  vi) Wait for acknowledgement from receiver
  vii) Reads a file called Receiver Buffer.
  viii) check ACK filled the acknowledgement number
  ix) If Acknowledgement number is as expected.

* Create a receiver file with following features:
  i) Read a file-called sender-buffer
  ii) Check frame number
  iii) If frame no. is as expected, write appropriate acknowledgement number in

receiver - buffer else write NACK number in receiver - buffer.

⇒ Sender code:

```
import os
import time
Base_dir = os.path.dir_name(os.path.abspath(
print("Sender buffer:", Sender_file)
print("Receiver buffer:", receiver_file)
window_size = int(input("Enter window size:"))
message = input("Enter text message:")
if window_size <=0:
    print("window size must be greater than
    false systemod;
frames[]
while start< total_frames:
    end = minstart + window_size, total_frame
    print("\n=====")
    print("Sending frames", Start+1, "to", end)
    print("\n=====")
elit_ack_type = "NACK"
    print("NACK received")
    print("Retransmitting from frames:", ack_no)
else:
    print("Unknown response: Retrying current window
print("\n=====")
print("Total frames", total_frames)
```

⇒ Receiver Program:

```
import time
import os
expected = 0
while true:
    if not os.path.exists("sender_buffer.txt"):
        time.sleep(1)
        continue
    with open("send_buffer.txt") as f:
        frames = f.readlines()
    if not frames:
        time.sleep(1)
        continue
    error = false
    for line in frames:
        n, data = line.strip().split(",")
        n = int(n)
        print("Received", n, data)
    if n != expected:
        with open("Receiver_buffer.txt", "w") as f:
            f.write("NACK:" + str(expected))
        error = true
        break
expected += 1
if not error:
    with open("Receiver_buffer_text", "w") as f:
        f.write("ACK" + str(expected))
open("sender_buffer_txt", "w").close()
time.sleep(1)
```

Sender code input:

window size = 3

message = HELLO

Sender Code output:

Frame 0: H

1: E

2: L

3: L

4: O

Receiver code Input:

Sender-buffer-text

0: H

1: E

2: L

3: L

4: O

Receiver Code Output:

Received: 0 H

1 E

2 L

3 L

4 O

* RESULT :

The Sliding Window Protocol was successfully implemented to control the frame flow at data Link Layer.

8/9/24

EXPT NO: 8
8/9/25

NMAP to Discover the Live Host using Nmap Scans (ARP, TCP/UDP) on the Tryhackme Platform

- **AIM:**

To discover Live Hosts using nmap Scans (ARP, TCP/UDP, TCMP) on the Tryhackme Platform

- **PROCEDURE:**

1. Setup and Environment Initialization:
   - Log in to TryHackme Platform and navigate to Target room: TryHackme Nmap Room
   - Deploy the Target virtual machine and establish a VPN connection using OpenVPN or Launch the browser based attach box.

2. ARP Scan for local Host Discovery:

   Perform an ARP Request scan to identify Live host on the Local network Segmet.

   Command:

   Sudo nmap -PR -sn target-IP <subnet>

3. TCMP Echo Ping Scan:

   Send TCMP Echo Request Packets to check for TCP Echo reply from Active host.

4. Perform TCMP Timerstamp Ping Scan.

Echo Request:

Command:

Sudo nmap -PP -sn <Target IP / Subnet>

5. Send TCP address mark request Packet to identify live systems

cmd:

Sudo nmap -PR -sn <Target - IP /Subnet>

6. Send TCP SYN Packets to Specified Ports to discover systems based on SYN ACK

cmd:

Sudo nmap -PS -sn <Target - IP/ Subnet>

7. TCP ACK Packets sent to receive RST response from active host

cmd:

Sudo nmap -PR -sn <Target IP/ Subnet>

8. Send empty UDP Packets to destination Ports to detect host via TCMP port unreachable response.

cmd:

Sudo nmap -PU -sn <Target - IP/ subnet>

9. Use the sn flag with nmap to perform host discovery exclusively without triggering subsequent Port Scans

cmd:

Sudo nmap -sn <Target -IP/ Subnet>

* RESULT:

Thus, live host discovery using various nmap scanning Techniques was successfully implemented on the TryHackme Platform.

8/9/

EXP NO: 9

24/9/24 Implementation of Subnetting in Cisco Packet Tracer Simulator

- AIM:

To implement classless IP Subnetting using CISCO PACKET TRACER and verify connectivity between different Subnets

- PROCEDURE:

① Open Cisco Packet Tracer and create a new network Topology

② Add routers, Switches and PCs to the workspace

③ Connect the devices using appropriate cables.

④ Divide the network 192.168.1.0/24 into smaller subnets using the /27 subnet mask.

⑤ Configure the IP addresses and subnet mask for the router interfaces.

⑥ Configure the PCs with suitable IP addresses, Subnet masks, & default gateway.

⑦ Configure the Switch Ports and router interface

⑧ Use the PING command to test connectivity between devices

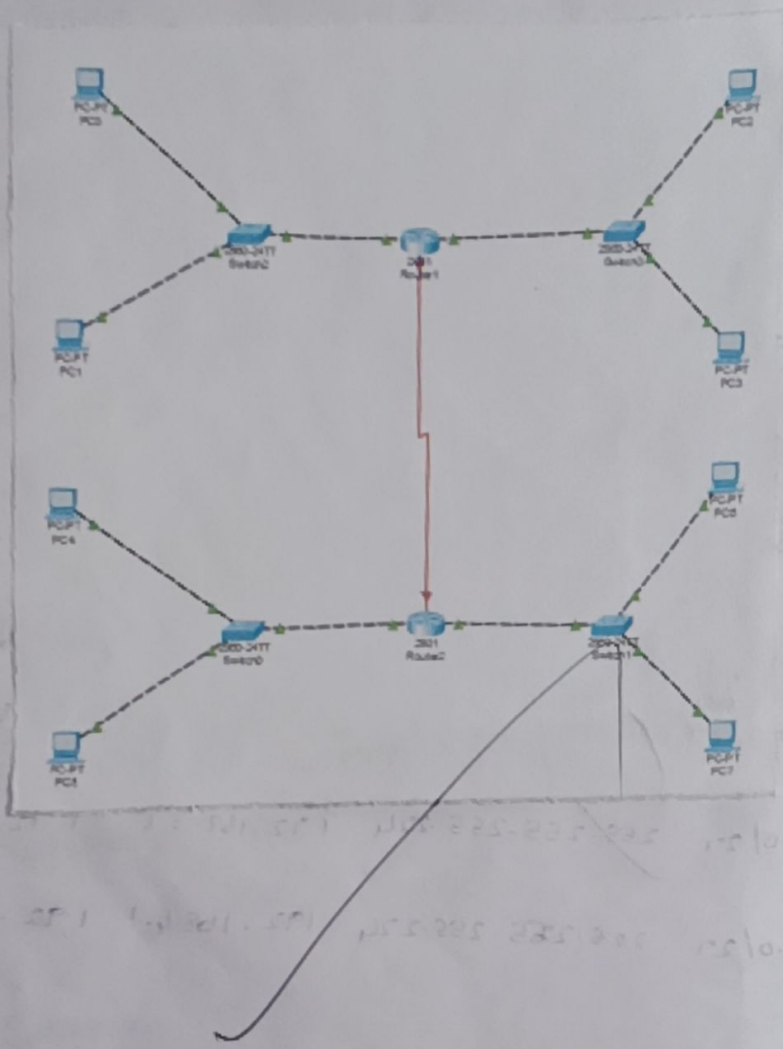
⑨ Verify the Packets are successfully Transmitted between the connected devices.

Advantages:

- Efficient use of IP Address
- Reduces network Congestion
- Improves network management.

IP Addressing

Network	Subnet Mask	Router IP	PC IP Range
192.168.1.0/27	255.255.255.224	192.168.1.1	192.168.1.11-15
192.168.2.0/27	255.255.255.224	192.168.2.1	192.168.2.11-15
192.168.3.0/27	255.255.255.224	192.168.3.1	192.168.3.11-15
192.168.4.0/27	225.255.255.224	192.168.4.1	192.168.4.11-15


PC-PT
PC0
PC-PT
PC2
2960-24TT
2960-24TT
PC-PT
PC1
PC-PT
PC3
PC-PT
PC4
PC-PT
PC5
2960-24TT
2801
2960-24TT
PC-PT
PC6
PC-PT
PC7

* RESULT :

The Classless IP Subnetting was successfully implemented using CISCO PACKET TRACER.

22/9

EXPT NO: 10.a

08/10/26

Internetworking with Router using CISCO Packet Tracer

- **AIM:**

To design and configure a simple internetwork using a router in CISCO Packet Tracer and verify connectivity btw 2 PC's.

- **OBSERVATION**

i) A Router and two PC's are used to create a simple Internetwork

ii) PC0 is connected to the 1st interface of Router1 using a copper straight-through cable

iii) PC1 is connected to the second interface of Router1

iv) Router Interfaces are assigned different network address.
- 192.168.10.1/24
- 192.168.20.1/24

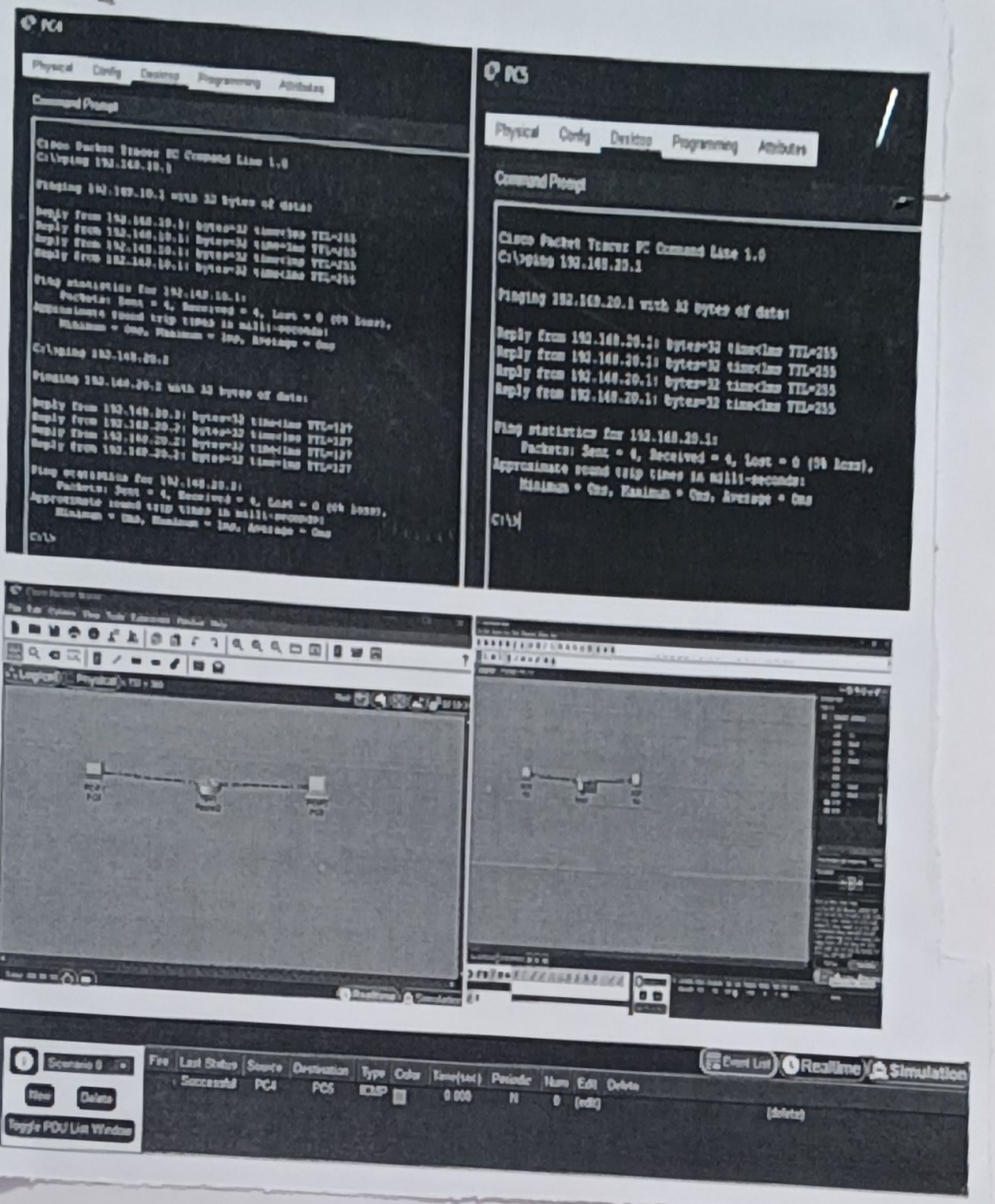
v) PC0 is assigned IP address 192.168.20.2 with gateway 192.168.20.1

vi) PC1 is assigned IP address 192.168.20.2 with gateway 192.168.20.1

vii) The Router forwards packets between two networks

viii) A PDU is sent from PC0 to PC1 to verify network connectivity.

EX 10 - A



- RESULT:

Thus, a simple Internetwork was successfully designed and configured using a router and communication btw PC0 and PC1 was successfully verified.

8/10

NO: 10 b
126

Wireless Router, DHCP Server and Internet Cloud.

AIM:

To design and configure an internetwork using a wireless router, DHCP Server and Internet ~~cable~~ cloud.

- OBSERVATIONS:

i) A Wireless Router, PC, Laptop, Cable modem, Internet Cloud and CISCO.com server are connected.

ii) The wireless router is configured with the SSID Home Network

iii) DHCP is enabled on the wireless router.

iv) The PC obtain its IPv4 address automatically through DHCP

v) The Laptop is configured with a wireless module and connected to home network.

vi) The network cloud is connected to the cable ~~router~~ modem

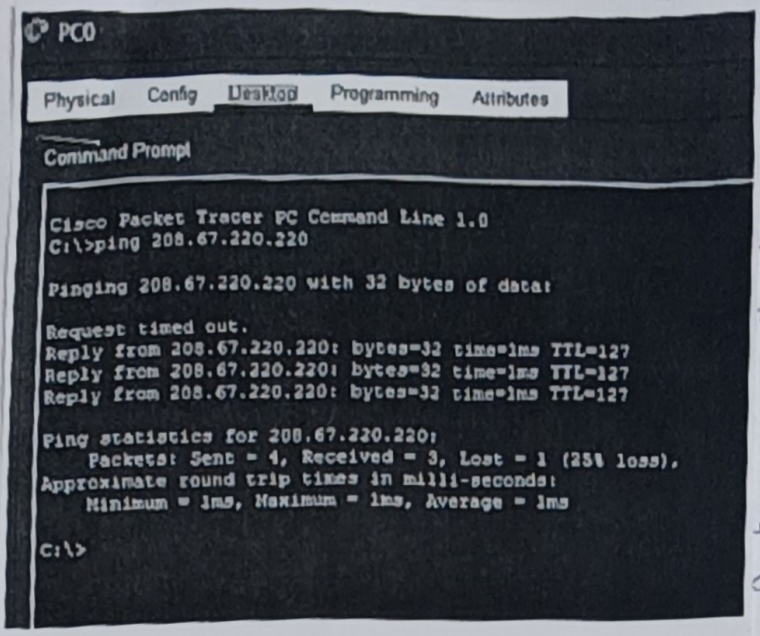
vii) The cisco.com server is configured a DHCP and DNS Server.

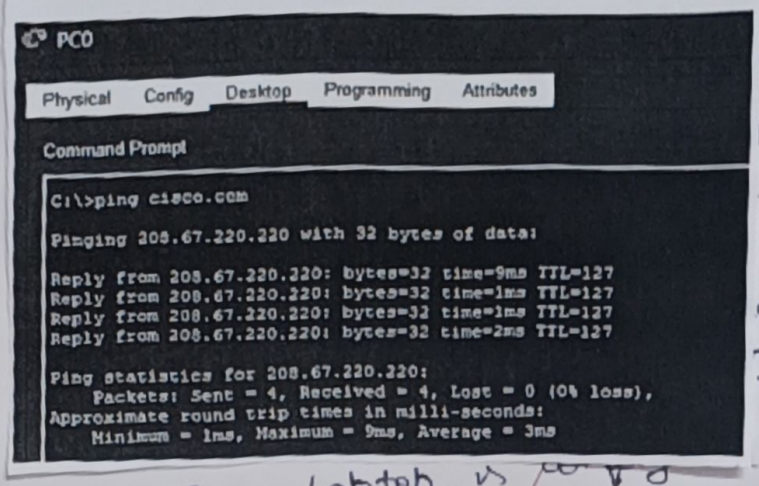
viii) The PC receives an address in the 192.168.0.x range.

ix) Connectivity is verified using ping cisco.com.

EXP NO: 10 b

05/10/26





erver and

l.

re an

reless router,

cloud.

aptop, cable

and cisco.com

, configured

Network

wireless

ddress

DHCP

d with a

v) The Laptop is [illegible] wireless module and connected to home network.

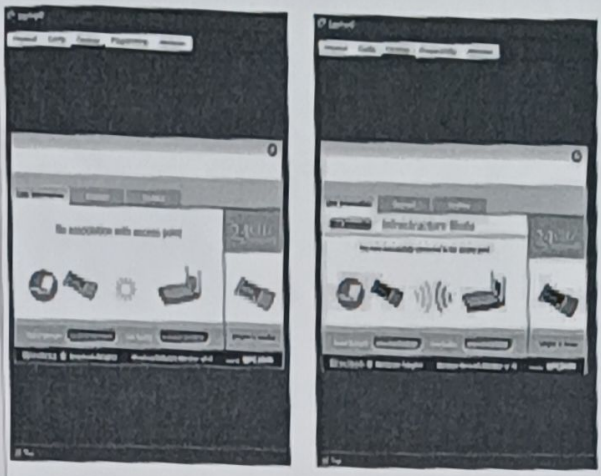
vi) The network cloud is connected to the cable modem

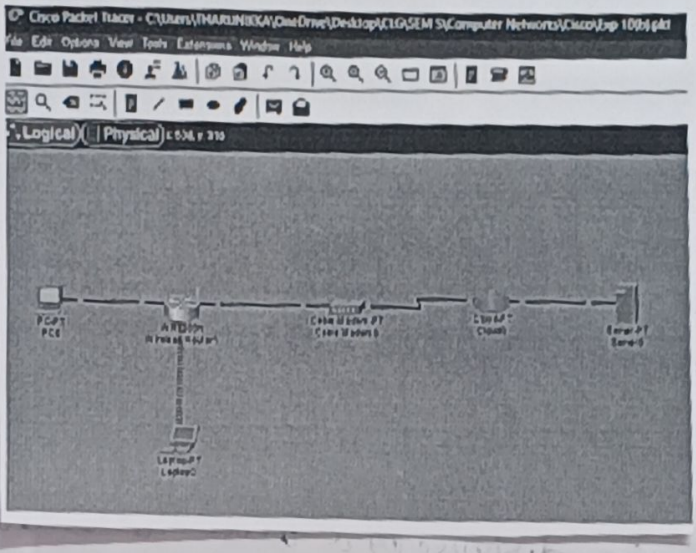
vii) The cisco.com server is configured as a DHCP and DNS server.

viii) The PC receives an address in the 192.168.0.x range.

ix) Connectivity is verified using ping cisco.com.

EX 10B





* RESULT:

Thus, the wireless router, DHCP server and Internet Cloud were successfully configured and connectivity to the cisco.com server was verified.

5/10/24

EXPT NO: 11a

08/10/26

STATIC ROUTING

- **AIM:**

To Simulate and configure static routing using CISCO Packet Tracer

- **OBSERVATION:**

i) Static Routing involves manually adding routes to a routes recording Table.

ii) Routers automatically knows the network directly connected to the interfaces.

iii) Routes are manually added for networks that are available through other routers.

iv) Both main and backup routes are configured for a destination.

v) Administrative distance (AD) is used to select the proffered static route.

vi) The routing table is check using: show ip route static

vii) Connectivity and routing paths are verified using ping and Tracer.

viii) Where the main route fails, the backup route can be used for forwarding Packets.

EXPT NO: 11a

06/10/26

STATIC ROUTING

- **AIM:**

To Simulate and configure static routing using CISCO Packet Tracer

- **OBSERVATION:**

i) Static Routing involves manually adding routes to a routes recording Table.

ii) Routers automatically knows the network directly connected to the interfaces.

iii) Routes are manually added for networks that are available through other routers.

iv) Both main and backup routes are configured for a destination.

v) Administrative distance (AD) is used to select the proffered static route.

vi) The routing table is check using:

:- route static

```
Router(config-if)#interface serial 0/1/1
Router(config-if)#no shutdown

Router(config-if)#
%LINK-5-CHANGED: Interface Serial0/1/1, changed state to up

%LINEPROTO-5-UPDOWN: Line protocol on Interface Serial0/1/1, changed state to up
```

```
C:\>tracert 30.0.0.100

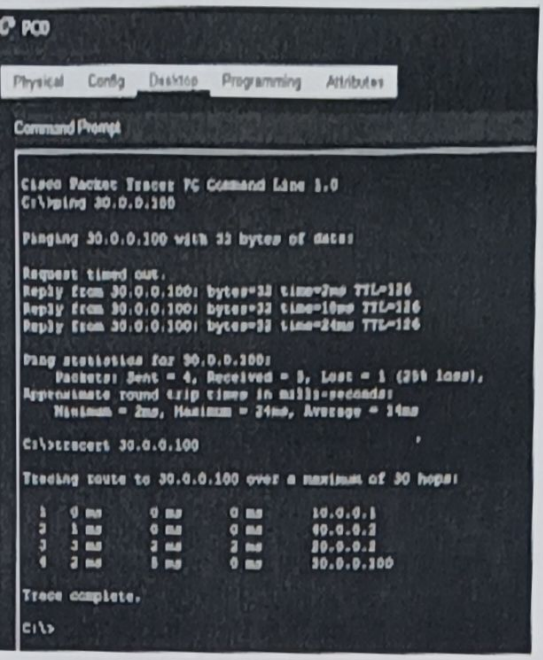
Tracing route to 30.0.0.100 over a maximum of 30 hops:

  1    0 ms      0 ms      0 ms      10.0.0.1
  2    1 ms      7 ms      4 ms      40.0.0.2
  3    9 ms      7 ms      1 ms      20.0.0.2
  4    2 ms      0 ms      4 ms      30.0.0.100

Trace complete.

C:\>
```

xp 11(a):




```
Router#enable
Router#configure terminal
Enter configuration commands, one per line.  End with CNTL/Z.
Router(config)#interface serial 0/1/1
Router(config-if)#shutdown

Router(config-if)#
%LINK-5-CHANGED: Interface Serial0/1/1, changed state to administratively down

%LINEPROTO-5-UPDOWN: Line protocol on Interface Serial0/1/1, changed state to down
```

```
C:\>tracert 30.0.0.100

Tracing route to 30.0.0.100 over a maximum of 30 hops:

  1    0 ms      0 ms      0 ms      10.0.0.1
  2    3 ms      1 ms      1 ms      20.0.0.2
  3    0 ms      0 ms      1 ms      30.0.0.100

Trace complete.
```

* RESULT:

Thus, Static routes were successfully configured and verified, including the use of backup route when the main route becomes unavailable

8/10/25

EXPT NO: 11b

08/10/25

RIP - Dynamic Routing

- AIM: To

To simulate and configure RIP [Routing Information Protocol] using Cisco Packet Tracer

- OBSERVATION:

i) Three routers are connected to form a network with multiple possible ~~packets~~ paths

ii) IP address are assigned to the PCs, Fast Ethernet Interfaces and serial Interfaces.

iii) RIP is enabled using the router RIP command.

iv) Directly connected networks are advertised using the network command

v) RIP automatically exchanges routing information between routers

vi) RIP selects the router based on the least hop count.

vii) ~~RIP selects the router based on the~~

vii) The Ping command is used to test connectivity between PC0 and PC1

viii) The Tracert command is used to verify the path followed by packets

ix) If the selected route fails, RIP automatically chooses another available routes.

EXPT NO: 11b

08/10/26

RIP - Dynamic Routing

- AIM:

To simulate and configure RIP [Routing Information Protocol] using CISCO Packet Tracer

- OBSERVATION:

i) Three routers are connected to form a network with multiple possible ~~packets~~ paths

ii) IP address are assigned to the PCs, Fast Ethernet Interfaces and serial Interfaces.

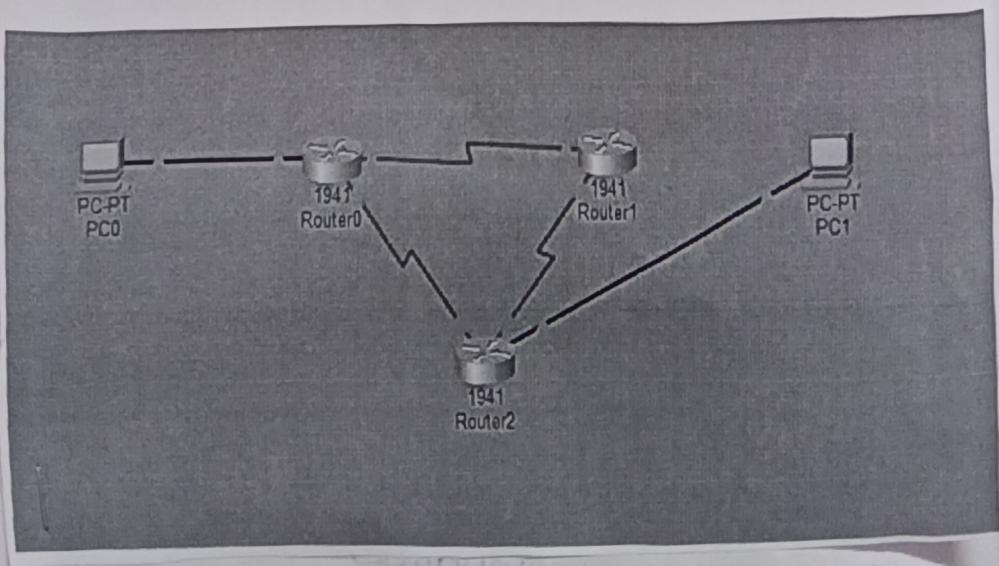
... the router RIP

```
C:\> tracert 20.0.0.2

Tracing route to 20.0.0.2 over a maximum of 30 hops:

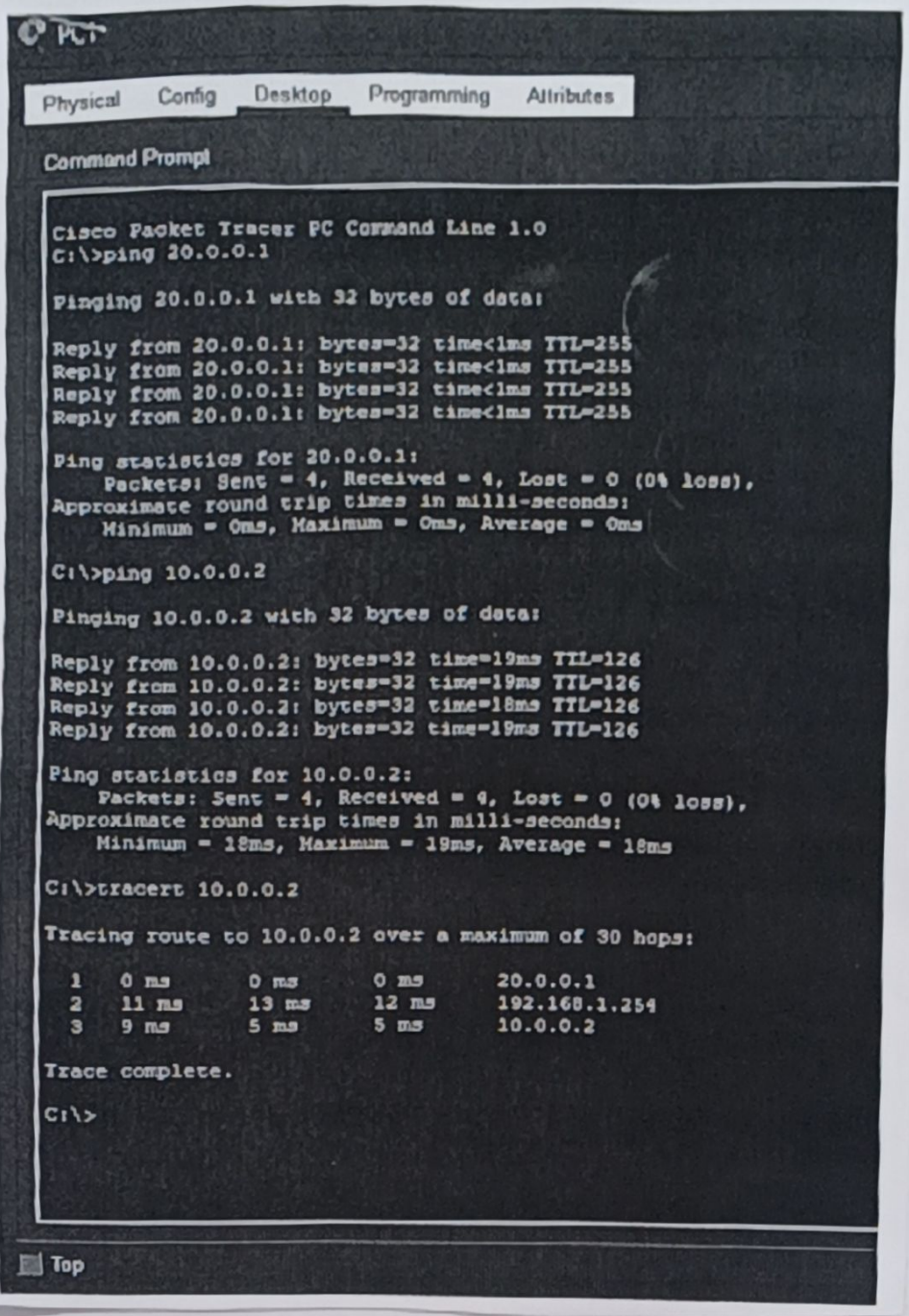
  1    0 ms      0 ms      0 ms      10.0.0.1
  2    6 ms      8 ms      7 ms      192.168.1.253
  3    5 ms      10 ms     7 ms      20.0.0.2

Trace complete.
```



routes.

Exp 11(b):



* RESULT:

Thus, RIP was successfully configured and dynamic routing was verified. The Router automatically selected the available route with the lower hop count.

EXPT NO: 12

29/9/24 END-TO-END COMMUNICATION AT TRANSPORT LAYER

(a) Implement ECHO client server using TCP sockets

- AIM:

To implement an ECHO SERVER application using TCP Sockets.

Code:

ECHO SERVER

```
import socket
s = socket.socket()
s.bind(("localhost", 5000))
s.listen(1)
c, addr = s.accept()
msg = c.recv(1024).decode()
print("Client:", msg)
c.send(msg.encode())
c.close()
s.close()
```

ECHO CLIENT

```
import socket
s = socket.socket()
s.connect(("localhost", 5000))
msg = input("Enter Message:")
s.send(msg.encode())
print("Echo:", s.recv(1024).decode())
s.close()
```

Input:

Enter message: Hello Server

Output:

Server
Client Connected
Received: Hello server
Client
Echo from Server: Hello Server.

RESULT:

The ECHO Client Server using TCP Socket was studied and executed Successfully.

29/9/24

(b) Implement chat client server using TCP/IP sockets

- AIM:

To implement a chat client server Application using TCP socket

- CODE:

SERVER

```
import socket
s=socket.socket()
s.bind(("localhost", 6000))
s.listen(1)
c, a = s.accept()
while True:
    msg = c.recv(1024).decode()
    print("client:", msg)
    if msg == "bye"
        break
    reply = input("server: ")
    c.send(reply.encode())
c.close()
```

CLIENT

```
import socket
s = socket.socket()
s.connect (("localhost", 5000))
while True:
    msg = input("Client:")
    s.send(msg.encode())
    if msg == "bye"
        break
    print("Server:", s.recv(1024).decode())
s.close()
```

Input:

Client: Hello
Server: Hi
Client: Bye

Output

Client: Hello
Server: Hi

* RESULT:

The chat client server Applications were successfully studied and implemented using TCP Sockets

29/9/26

EXPT NO: 13

30/9/26

Implement your own PING Program

- **AIM:**

To implement a Ping Program using Python to check the connectivity between source and destination host.

- **ALGORITHM:**

i) Get the Destination IP address from users

ii) Send the request to destination

iii) Wait for response.

iv) Calculate the response Time

v) Display whether the host is reachable.

vi) Repeat the process for few request.

CODE

```
import os
import time
host = input("Enter host:")
for i in range(4):
    start = time.time()
    result = os.system("ping -n 1 " + host +
                       " > nul")
    end = time.time()
    if result == 0:
        print("Reply from", host, "Time=",
              round((end - start) * 1000, 2), "ms")
    else:
        print("Request timed out")
    time.sleep(1)
```

INPUT

Enter host: 127.0.0.1

OUTPUT

Reply from 127.0.0.1 Time = 1.23 ms

Reply from 127.0.0.1 Time = 0.98 ms

RESULT :

The Ping Program was successfully studied and implemented.

30/9

EXPT NO: 14

01/10/24

Packet Sniffing Using Raw Source

- **AIM:**

To write a Program using RAW sockets to capture and display network Packets.

- **ALGORITHMS:**

i) Create a RAW Socket

ii) Bind the socket to the network Interface.

iii) Receive Packets from the network.

iv) Extract the IP header information.

v) Display the source IP, destination IP and Protocol.

vi) continue capturing ~~Protocol~~ Packets

CODE

```
import socket
s = socket.socket(socket.AF_INET, socket.
        SOCK_RAW, socket.IPPROTO_IP)
s.bind(("0.0.0.0", 0))
s.setsockopt(socket.IPPROTO_IP,
        socket.IP_HDRINCL, 1)
print("Packet Sniffing Started...")

while True:
    data, addr = s.recv from (65535)
    print("Packet Received from :", addr[0])
```

Input

Network Packets

Output

```
Packet Sniffing Started...
Packet Received from : 192.168.1.5
Packet Received from : 192.168.183.14
Packet Received from : 192.168.120.183.14
```

* RESULT

The RAW SOCKET SNIFFING PROGRAM was successfully implemented and network Packets was captured and displayed.

01/10/24